

ХАКАТОН · ЗАДАЧА 2

Умный роутинг выплат

Движок распределения выплат между платёжными провайдерами:
допуск, выбор, каскад, объяснение и рекомендации

Команда «Коломот» · Фёдор Филитович · Артём Корсаков

Ruby 3.3 · 0 внешних зависимостей · 122 теста · github.com/T0vich/kolomot-smart-payout-routing

«Выбрать подходящего партнёра» — это три разных вопроса

01

Кому **можно**?

Ответ бинарный. Лимиты, банки, реквизиты, маржа. Нарушать нельзя никогда — независимо от того, какая стратегия включена.

02

Кого **лучше**?

Ответ — всегда компромисс. Семь целей из ТЗ конфликтуют между собой, и это не баг постановки, а её суть.

03

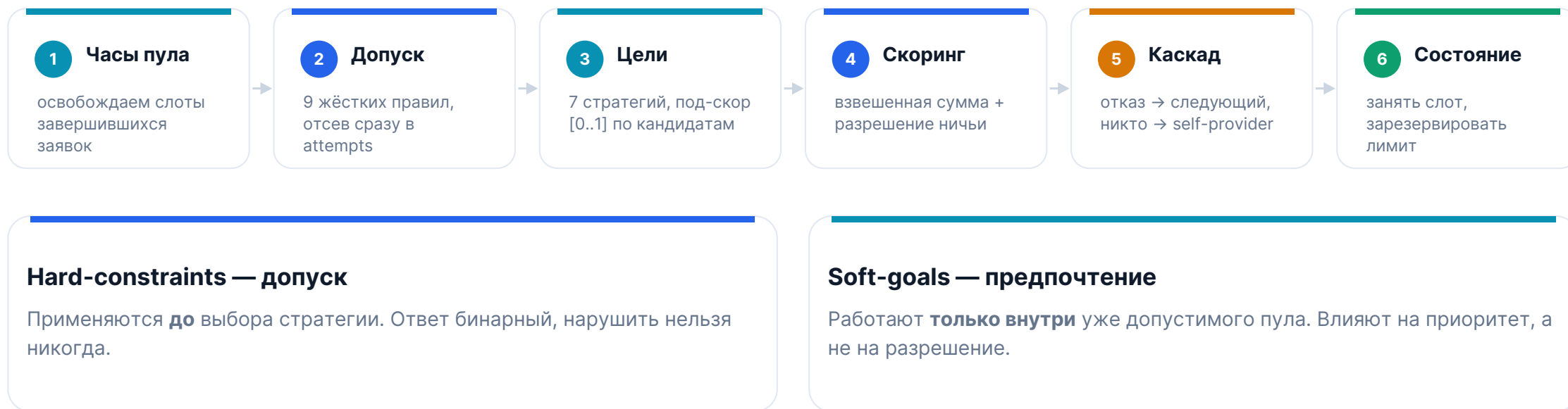
Что если **отказал**?

Нужна последовательность попыток, которая ни на одном шаге не нарушает жёсткие ограничения.

Пример конфликта прямо из ТЗ. Цель — 40 % заявок на вірау. Факт — вірау почти исчерпал дневной максимум, а на payflow висит обязательство «не менее 2 млн ₽ в сутки». Обе цели корректны. Одновременно выполнить нельзя. Кому отдаём заявку — и как это объяснить?

Большинство ошибок в таких системах — от попытки решить все три вопроса одним механизмом.

Два слоя, которые нельзя смешивать



Стратегия физически не может «угговорить» роутер нарушить лимит. Альтернативу — единый скоринг со штрафом за нарушение — мы отвергли: там нарушение перестаёт быть невозможным и становится просто дорогим.

Девять правил допуска и семь целей — все из ТЗ

ДОПУСК · 9 ЖЁСТКИХ ПРАВИЛ · НАРУШИТЬ НЕЛЬЗЯ

- Статус провайдера**
provider_inactive
- Диапазон суммы чека**
amount_below_minimum · amount_exceeds_limit
- Дневной максимум**
daily_limit_exceeded
- Одновременные заявки**
in_progress_count_exceeded · in_progress_amount_exceeded
- Банковский фильтр**
bank_not_in_list · bank_in_exclude_list
- Маржа**
negative_margin
- Реквизиты**
no_requisites
- Интенсивность**
rate_limit_exceeded
- Потолок оборота — наше дополнение**
turnover_max_reached

ПРЕДПОЧТЕНИЕ · 7 ЦЕЛЕЙ · КОМПРОМИСС

- | | | |
|---|--------------------|---------------------------|
| 1 | Доля по количеству | traffic_percentage |
| 2 | Доля по объёму | volume_share_pct |
| 3 | Очередь в каскаде | priority |
| 4 | По сумме чека | amount_bands |
| 5 | По конверсии | conversion_24h |
| 6 | По интенсивности | requests_per_minute_limit |
| 7 | Фин. обязательства | daily_turnover_min / max |

```
{ "provider": "payflow", "decision": "skipped", "stage": "hard_filter",
  "reason": "amount_exceeds_limit", "details": "150000 > limit_amount_max 50000" }
```

Диапазон суммы и загрузка влияют на выбор, а не только отсекают: жёсткая версия живёт в допуске, мягкая — в целях. Всё настраивается **config/routing.yml**.

Учитываем всё сразу — и разрешаем конфликт явно

$$\text{score(провайдер)} = \sum (\text{под-скор}_i \times \text{вес}_i) / \sum \text{вес}_i$$

Веса профиля `balanced`, сдаваемого по умолчанию:



1 Разрыв баллов мал

Явный порядок политик: фин. обязательства → доля по количеству → конверсия → доля по объёму → каскад → диапазон суммы → загрузка

2 Цели равны полностью

Детерминированный разрыв: `priority`, затем имя. Один вход всегда даёт один выход — это отдельный тест.

3 Цель невыполнима

Нужный провайдер отсечён допуском. Цель **не блокирует** роутинг: заявка уходит дальше, расхождение — в отчёт с рекомендацией.

Почему обязательства выше долей. За недобор минимального оборота платят деньгами по договору, а целевая доля — ориентир, её можно добрать следующими заявками. Веса и порядок политик живут в конфиге: приоритет бизнеса меняется одной строкой, а не правкой кода.

Почему выбран он, почему не остальные и что при отказе

op_101 · 15 000 ₽ · Сбербанк. Допустимы все трое, и цели расходятся: по диапазону суммы лучше payflow, по загрузке — quickpay, по долям и конверсии — vipay.



Отказ провайдера принять заявку — это каскад. Профиль demo_failover, op_103 на 150 000 ₽:

```
vipay      skipped  amount_exceeds_limit  150000 > limit_amount_max 100000
payflow    skipped  amount_exceeds_limit  150000 > limit_amount_max 50000
quickpay   skipped  provider_declined      провайдер отказался принять заявку
spacepayments selected fallback_self_provider  внешних допустимых не осталось
```

Выплата, которая не прошла у принявшего провайдера, — терминальный исход, маршрут она не меняет. Противоположное прочтение включается флагом **retry_on_terminal_failure**: реализован, покрыт тестами, по умолчанию выключен.

Лимиты резервируем, а не считаем пост-фактум

Наивная реализация

`daily_approved_amount` растёт только после подтверждения выплаты.

Десять заявок, отправленных подряд, все проходят проверку дневного лимита — и лимит продан десять раз.

Наша реализация

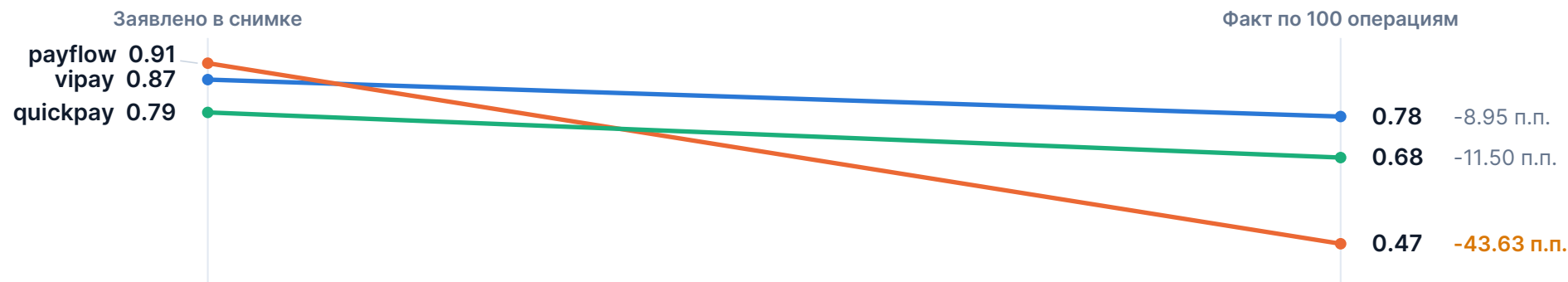
Сумма резервируется в момент выбора провайдера:

`занято = daily_approved_amount + daily_reserved_amount`

При завершении резерв снимается. То же для in-progress count и amount, плюс модельное время: заявка держит слот `latency_sec` и освобождает его.

Побочная выгода. Валидатор организаторов считает лимиты по исходному снимку `providers.json`. Наша проверка строго жёстче — значит, выбранный нами провайдер гарантированно входит в его список допустимых. Результат на публичной очереди: **29 проверок пройдено, 0 ошибок, 0 предупреждений.**

Отчёт называет причину расхождения и параметр к изменению



32 → 26 п.п.

суммарное отклонение от целевых долей на 100 исторических операциях

29 из 100

исторических выплат ушли провайдеру, который по правилам допуска не мог их принять

26 vs 38 п.п.

balanced против conversion_first при конверсии 0.66 и 0.69 — цена политики

Главная находка. Провайдер с самой высокой заявленной конверсией на деле худший: payflow обещает 0.91, по истории даёт 0.47. Поэтому конверсия у нас смешанная. Рекомендации называют провайдера, число и параметр: «payflow выбрал 98.29 % дневного лимита — поднять daily_amount_limit до 3 600 000 ₽ или снизить долю трафика».

Что меняется без кода — и как всё проверить

122

теста, 796 проверок

29 / 29

проверок валидатора

0

внешних зависимостей

7

профилей роутинга

CI

зелёный на каждый push

МЕНЯЕТСЯ БЕЗ ЕДИНОЙ СТРОКИ КОДА

- Новый провайдер — строка в providers.json
- Лимиты, банки, конверсия, целевые доли — providers.json
- Приоритеты бизнеса — веса профиля в конфиге
- Порядок разрешения конфликтов — ключ conflict_order
- Целиком другая политика — новый профиль и флаг --profile

ПРОВЕРИТЬ САМИМ

- | | |
|-------------------------------------|----------------------------|
| Тесты без внешних зависимостей | <code>rake test</code> |
| Прогон публичной очереди | <code>rake route</code> |
| Валидатор организаторов | <code>rake validate</code> |
| Каскад и fallback в действии | <code>rake demo</code> |
| Сдаваемые файлы из тестовой очереди | <code>rake submit</code> |

Готовы к вопросам

Ограничения кейса соблюдены: нейросетей внутри решения нет, проприетарных компонентов нет, весь код — Ruby на стандартной библиотеке. Повторный запуск на тех же данных даёт побайтово тот же результат.

Почему веса именно такие

- Веса — не подобранная магия, а расстановка приоритетов, которую можно защитить словами. **Доли по количеству и объёму вместе дают 0.42** — это прямое требование ТЗ и договора.
- **Конверсия 0.18** — второй по величине одиночный вес: отправлять туда, где выплата не проходит, дороже, чем немного отклониться от доли.
- **Фин. обязательства 0.14** — вес умеренный, потому что стратегия и так разрывная: она выстреливает только когда минимум не набран.
- **Каскад, диапазон суммы, загрузка — 0.26 на троих**: это уточняющие сигналы, жёсткие версии этих же ограничений уже отработали в слое допуска.
- Проверка на истории: профили **traffic**, **volume**, **commitments** и **balanced** дают одинаковые 26 п.п. Результат устойчив к перекосу весов, а не держится на одной подобранной комбинации.

Нормировка: сравнительные и абсолютные сигналы

Сравнительные — min-max по пулу

traffic_share, volume_share, conversion, cascade_priority.

Разброс конверсий в кейсе — 0.79...0.91. Без нормировки эта разница растворяется среди остальных слагаемых и не влияет ни на что.

Абсолютные — по своей шкале

load_balance, turnover_commitment, amount_band.

«Загружен на 64 %» осмысленно само по себе: сравнивать не с чем и не нужно.

Следствие, о котором стоит сказать честно. При двух кандидатах сравнительные стратегии дают ровно 1.0 и 0.0, и выбор превращается во взвешенное голосование. На публичной очереди из 10 заявок это видно: 4 заявки безальтернативны, на остальных шести все содержательные профили сходятся к одному решению. Расхождение политик проявляется на объёме — поэтому у нас есть backtest.

Обработка ошибок и граничных ситуаций

Ситуация	Поведение
Файл не найден, битый JSON	Ошибка с путём и причиной разбора, а не стектрейс
amount строкой, нулём, отрицательным; заявка без id	Отклоняется на загрузке с указанием поля и значения
Дубли operation_id	Отклоняются: молча смаршрутизировать дубль хуже, чем упасть
Очередь — один объект вместо массива	Принимается: валидатор организаторов допускает оба варианта
Пустая очередь	Валидный отчёт с нулями, без деления на ноль
providers.json не массив / без обязательного поля	Ошибка с указанием, чего именно не хватает
Неизвестная стратегия или правило в конфиге	Ошибка при старте, а не тихое игнорирование
Под заявку не подходит вообще никто	Уходит на spacepayments, помечается fallback_used
daily_amount_limit = null	Отсутствие лимита, а не ноль
У провайдера кончились реквизиты посреди очереди	Покидает пул, остальные заявки идут дальше

Всё перечисленное закрыто тестами: 11 в loaders_test.rb, 9 в config_test.rb, 18 в report_test.rb, 17 в router_test.rb.

Спорные места Т3: что выбрали и что переключается

Вопрос	Наша трактовка	Если прочтение другое
Неуспешная выплата — отказ, требующий перемаршрутизации, или финал?	Финал. К следующему переходим, только если провайдер не принял заявку в работу	Флаг <code>retry_on_terminal_failure</code> — реализован, покрыт тестами, по умолчанию выключен
Скрытая проверка считает лимиты по исходному снимку <code>providers.json</code> ?	Да — поэтому мы намеренно строже: резервируем лимит в момент выбора	Ослабить резервирование в конфиге; менее строгий вариант нам не нужен
Запрет нейросетей — про решение или про написание кода?	Про решение: внутри проекта моделей нет, только правила и арифметика	Ничего: решение не вызывает ни одной модели и ни одного внешнего сервиса
Можно ли добавлять свои поля в решение?	Да, Т3 прямо разрешает дополнительные поля	Убрать <code>score</code> , <code>score_breakdown</code> , <code>stage</code> , <code>eligible_providers</code> из <code>Decision#to_h</code>

Отдельной QA-сессии с ответами организаторов не было, поэтому по каждому спорному месту мы выбрали более строгий вариант, записали обоснование в **docs/QA-ANSWERS.md** и заранее подготовили переключатель.